Adapting Methods for Domain-Specific Japanese Small LMs: Scale, Architecture, and Quantization

Takato Yasuno

Abstract

This paper presents a systematic methodology for building domain-specific Japanese small language models using QLoRA fine-tuning. We address three core questions: optimal training scale, base-model selection, and architecture-aware quantization.

Stage 1 (Training scale): Scale-learning experiments (1k–5k samples) identify n=4,000 as optimal, where test-set NLL reaches minimum (1.127) before overfitting at 5k samples. **Stage 2 (Compare finetuned SLMs):** Comparing four Japanese LLMs shows that Llama-3 models with Japanese continual pre-training (Swallow-8B, ELYZA-JP-8B) outperform multilingual models (Qwen2.5-7B). **Stage 3 (Quantization):** Llama-3 architectures improve under Q4_K_M quantization, while GQA architectures degrade severely (Qwen2.5: -0.280 points).

Production recommendation: Swallow-8B Q4_K_M achieves 2.830/3 score, 8.9 s/question, 4.9 GB size. The methodology generalizes to low-resource technical domains and provides actionable guidance for compact Japanese specialist LMs on consumer hardware.

Keywords: QLoRA Fine-Tuning, Scale Learning, Model Quantization, Japanese LLM, Llama-3, Grouped-Query Attention, Domain Adaptation, LLM-as-Judge, Construction Standards, Swallow, Qwen2.5, Technical NLP.

1 Introduction

1.1 Background and Motivation

Domain-specific adaptation of large language models (LLMs) has emerged as a critical requirement for deploying AI systems in specialized technical fields such as construction engineering, medical diagnosis, and legal consultation [1, 2]. While general-purpose LLMs trained on broad web corpora demonstrate impressive capabilities, they systematically underperform on domain-specific tasks requiring precise knowledge of technical standards, regulatory requirements, and field-specific terminology.

Parameter-efficient fine-tuning (PEFT) methods, particularly QLoRA (Quantized Low-Rank Adaptation) [3], have democratized domain adaptation by enabling fine-tuning of multi-billion-parameter models on consumer-grade GPUs through 4-bit quantization and low-rank weight updates [4]. However, practical deployment of QLoRA-adapted models requires answering three fundamental engineering questions that remain underexplored in the literature:

1. **Training scale:** How much domain-specific training data is necessary and sufficient? Beyond what point does additional data yield diminishing returns or induce overfitting?
2. **Model selection:** How do different Japanese LLM architectures (standard Multi-Head Attention vs. Grouped-Query Attention, diverse tokenization strategies) respond to identical QLoRA training on a low-resource technical corpus?
3. **Quantization:** Does aggressive quantization (e.g., Q4_K_M GGUF) after QLoRA training degrade quality? Are effects consistent across architectures, or do certain designs exhibit architecture-specific vulnerabilities?

This paper addresses these questions through a **three-stage progressive optimization methodology (Stages 1–3)** applied to the domain of Japan's River and Sediment Control Technical Standards—a multi-volume regulatory corpus governing civil infrastructure design and maintenance.

1.2 Experimental Stages Overview

Stage 1: Training Scale (Section 5). We train five QLoRA models with dataset sizes n = 1,000, 2,000, 3,000, 4,000, and 5,000 on Swallow-8B-Instruct [5] using LoRA rank r=16. Training loss decreases monotonically with scale, but test-set negative log-likelihood (NLL) analysis reveals **n=4,000 as the optimal point** (NLL 1.127, perplexity 3.09),

with overfitting emerging at n=5,000 (NLL 1.319, +17% increase). This provides empirical guidance for efficient data curation in low-resource domains.

Stage 2: Compare Finetuned SLMs (Section 6). Using the optimal n=4,000 dataset, we compare four Japanese LLMs: Swallow-8B (Llama-3, Japanese continuation pre-training), ELYZA-JP-8B (Llama-3, Japanese instruction-tuned), Qwen2.5-7B (GQA architecture, multilingual), and Tanuki-8B (Llama-3, llm-jp tokenizer). All models undergo identical QLoRA training (r=32, alpha=16, dropout=0.05) and are evaluated on a 100-question benchmark using Qwen2.5-14B as an LLM judge. **Key finding:** Japanese continuation pre-training (Swallow-8B: 2.820/3, 84% perfect-score rate) provides a decisive advantage over multilingual models (Qwen2.5-7B: 2.420/3, 49%) in this technical domain.

Stage 3: Quantization (Section 7). We systematically compare F16 (merged_16bit safetensors) and Q4_K_M GGUF quantization for all four models. **Surprising discovery:** Llama-3 models (Swallow, ELYZA) show *improved* quality under Q4 quantization (+0.010 to +0.030 points), likely due to regularization suppressing overfitting on the n=4,000 dataset. In contrast, Qwen2.5-7B's GQA architecture suffers severe degradation (F16 2.420 → Q4 2.140, -0.280 points), indicating that shared key-value representations in GQA are vulnerable to quantization noise. Inference speed improves 2.5–6.1× with Q4, making quantized deployment essential for production systems.

1.3 Contributions

1. **Empirical scale-learning methodology:** We demonstrate that test-set NLL analysis can identify the optimal training scale, preventing both underfitting (n < 4,000) and overfitting (n > 4,000) in domain adaptation scenarios. Our findings suggest that for Japanese technical QA datasets of similar complexity, 4,000 well-curated examples provide a practical target.

2. **Architecture-dependent quantization effects:** We document for the first time that Q4 quantization can *improve* Llama-3 model quality post-QLoRA through regularization, while simultaneously degrading GQA architectures. This has immediate implications for production deployment: Llama-3 models can be safely quantized to Q4_K_M, while GQA models require F16 or Q8.

3. **Multi-model comparative analysis:** By evaluating four diverse Japanese LLMs (Llama-3 standard MHA, Llama-3 ELYZA variant, Qwen2.5 GQA, Tanuki Llama-3 with llm-jp tokenizer) under identical training and evaluation conditions, we isolate the impact of architectural choices, pre-training corpus language composition, and tokenization strategy on domain adaptation performance.

4. **Complete reproducible pipeline:** We document end-to-end engineering details including data rebalancing (HAS_CHAPTER undersampling from 24.5% to 17.1%), unsloth training configuration, llama.cpp quantization workflow, Ollama deployment, and LLM-Judge rubric design, enabling replication on consumer hardware (single RTX 4060 Ti 16 GB).

5. **Five actionable engineering lessons:** We distill practical insights spanning scale selection (test-set NLL analysis), model selection (Japanese-focused pre-training advantages), quantization strategies (architecture-dependent effects), training configuration (LoRA rank optimization), and data curation (relation-type balancing). These lessons address recurring pitfalls encountered across Stages 1–3, providing a checklist for practitioners deploying QLoRA in low-resource technical domains.

1.4 Paper Organization

Section 2 reviews related work. Section 3 formalizes the task and evaluation framework. Section 4 describes dataset construction and rebalancing. Section 5 presents the Stage 1 scale-learning experiments. Section 6 reports the Stage 2 multi-model comparison. Section 7 analyzes quantization effects and architecture dependencies. Section 8 distills engineering lessons learned. Section 9 discusses implications and limitations. Section 10 concludes.

Figure 1 provides a visual overview of the three-stage methodology.

2 Related Work

2.1 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [4] introduces trainable low-rank decomposition matrices $\Delta W = BA$ (where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll \min(d,k)$) into frozen pre-trained weights, reducing trainable parameters from billions to millions while maintaining competitive performance on downstream tasks.

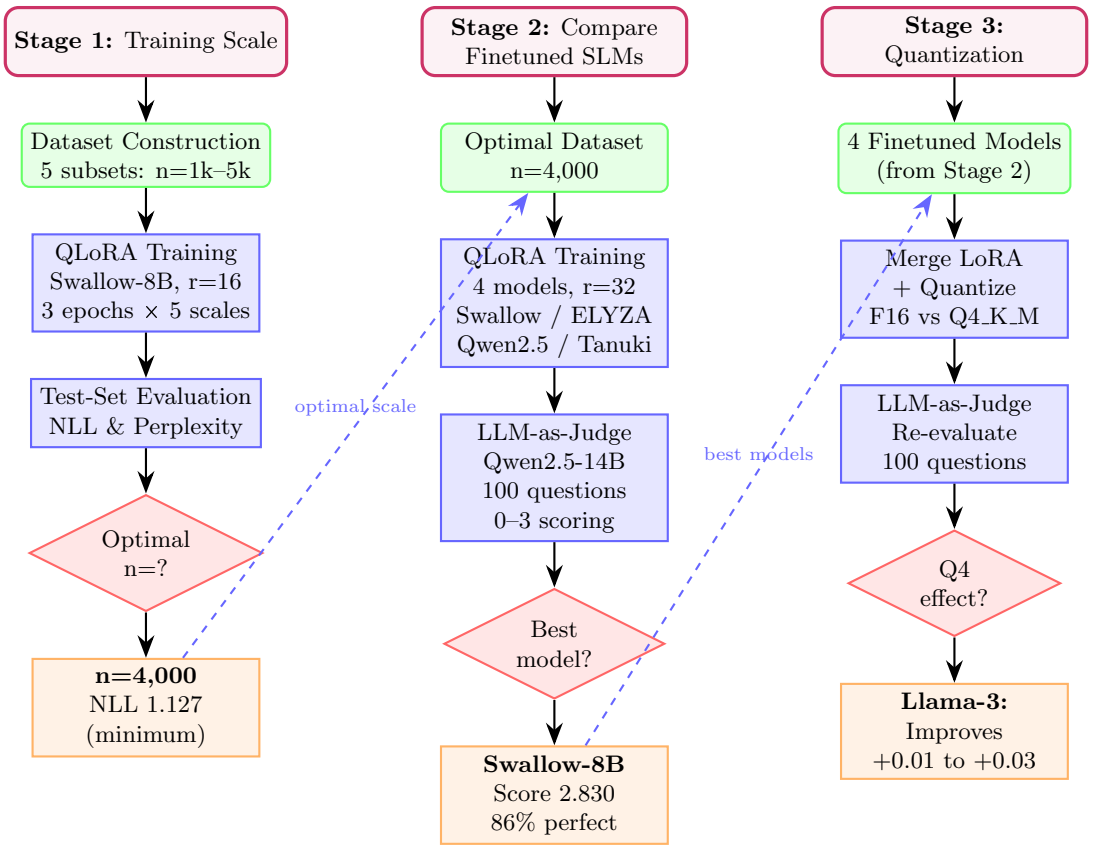

Figure 1: Three-stage progressive optimization methodology. Stage 1 identifies optimal training scale (n=4,000) via test-set NLL analysis across five dataset sizes. Stage 2 compares four Japanese LLMs under identical QLoRA training, revealing Japanese continual pre-training advantage (Swallow-8B: 2.830/3). Stage 3 discovers that Llama-3 models improve under Q4_K_M quantization (+0.01 to +0.03 points). Dashed arrows indicate data flow between stages.

QLoRA [3] extends this paradigm by quantizing base model weights to 4-bit NormalFloat (NF4) precision and performing backpropagation through quantization-aware adapters, enabling fine-tuning of 70B models on consumer GPUs. Recent advances include weight-decomposed low-rank adaptation (DoRA) [6], adaptive budget allocation for parameter-efficient tuning (AdaLoRA) [7], and dynamic search-free adaptation (DyLoRA) [8]. Subsequent work has explored gradient-based quantization [9], memory-efficient gradient projection [10], and unified frameworks for diverse PEFT methods [11]. `unsloth` [12] provides optimized kernels achieving 2–5× speedup and 70% memory reduction for QLoRA training.

Our work extends this line by empirically investigating *how much* training data is necessary for QLoRA to be effective (scale learning) and *whether* aggressive post-training quantization undermines adaptation quality (quantization analysis).

2.2 Model Quantization

Post-training quantization compresses model weights to lower bit-widths (INT8, INT4) to reduce memory footprint and accelerate inference [13]. GPTQ [14] and AWQ [15] employ calibration datasets to minimize quantization error layer-by-layer. Recent methods include SmoothQuant [16] for accurate post-training quantization, QuIP [17] for 2-bit quantization with guarantees, BitNet [18] for scaling 1-bit transformers, and OmniQuant [19] for omnidirectionally calibrated quantization. GGUF (GPT-Generated Unified Format) standardizes quantized model distribution, with K-quantization variants (Q4_K_M, Q5_K_S) mixing 4-bit and 6-bit representations across layer types [20].

While prior work primarily evaluates quantization on zero-shot benchmarks using *pre-trained* models, we investigate quantization applied to *QLoRA-adapted* models, revealing architecture-dependent effects not previously documented.

2.3 Japanese Language Models

The Llama 3 architecture [21] serves as the foundation for numerous Japanese adaptations. The Swallow family [5] performs continual pre-training on Japanese web corpora with vocabulary expansion and learning rate warmup [22]. ELYZA-JP-8B is a commercially-released instruction-tuned variant optimized for concise generation. Qwen2.5 [23] introduces Grouped-Query Attention (GQA) for efficient long-context modeling, trained multilingually. Tanuki-8B employs the Llama-3 8B architecture with llm-jp tokenizer v2.1, optimized for Japanese text processing. LLM-jp [24] and Rinna [25] represent significant efforts in developing open Japanese language models with dedicated tokenizers and pre-training corpora.

We compare these four models under identical fine-tuning conditions to isolate the impact of architecture and pre-training strategy.

2.4 LLM Evaluation and LLM-as-Judge

MT-Bench [26] formalizes the LLM-as-Judge paradigm, demonstrating high correlation with human evaluation on multi-turn dialogue tasks. Recent evaluation frameworks include AlpacaEval [27] for automatic evaluation of instruction-following models, AlpacaFarm [28] for simulating human feedback, Chatbot Arena [29] for human preference evaluation, and the Open LLM Leaderboard [30] for standardized benchmarking.

We adopt a 0–3 scoring rubric judged by Qwen2.5-14B, emphasizing citation of specific technical standards (score 3), correctness without citations (score 2), partial correctness (score 1), and incorrect or missing answers (score 0).

2.5 Instruction Tuning and Domain Adaptation

Instruction tuning aligns pre-trained language models with human preferences through supervised fine-tuning on instruction-response pairs. InstructGPT [31] demonstrates that training with human feedback significantly improves model helpfulness, harmlessness, and honesty. Llama 2 [32] combines supervised fine-tuning with reinforcement learning from human feedback (RLHF) to create open-access chat models. LIMA [33] shows that alignment can be achieved with surprisingly small high-quality instruction datasets (less is more). Crosslingual fine-tuning [34] extends instruction-following capabilities across multiple languages through multitask learning.

Our work applies QLoRA to domain-specific instruction datasets (dam management standards), investigating the intersection of parameter-efficient adaptation with specialized knowledge acquisition.

3 Problem Formulation

3.1 Task Definition

Input: A natural-language question q in Japanese about the River and Sediment Control Technical Standards.

Output: A natural-language answer a that is technically accurate, specific, and grounded in the standards.

Corpus: The standards comprise four volumes (Survey, Planning, Design, Maintenance) covering survey protocols, risk assessment, structural design criteria, inspection intervals, and emergency response procedures for river levees, dams, sabo (torrent control) structures, and landslide prevention facilities.

3.2 Evaluation Protocol

We construct a 100-question test set (Section 4) covering 8 technical categories: Survey, Planning, Design, Maintenance-River, Maintenance-Dam, Maintenance-Sabo, Hazard, and Comparative/Cross-Cutting.

Each question q_i is answered by model $\mathcal{M}$, producing answer a_i. An independent judge model $\mathcal{J}$ (Qwen2.5-14B via Ollama) scores each answer on a 0–3 rubric:

- **3 points:** Technically accurate, specific, and includes citation of standards (e.g., volume name, chapter number, technical term).
- **2 points:** Technically correct but lacks supporting evidence or specificity.
- **1 point:** Partially correct; contains important errors or omissions.
- **0 points:** Incorrect, irrelevant, or no answer provided.

Metrics:

- **Average Score:** $\bar{s} = \frac{1}{100}\sum_{i=1}^{100} s_i$ where $s_i \in \{0, 1, 2, 3\}$.
- **Perfect-Score Rate:** Percentage of questions scored 3.
- **Inference Time:** Average seconds per question.
- **Response Length:** Average number of Japanese characters.

3.3 Mathematical Formulation

QLoRA Fine-Tuning. Given a pre-trained language model with frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$, Low-Rank Adaptation (LoRA) [4] introduces trainable low-rank matrices $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$ where $r \ll \min(d, k)$:

$$h = W_0 x + \frac{\alpha}{r} BAx \tag{1}$$

where α is a scaling hyperparameter and $x \in \mathbb{R}^k$ is the input activation. During training, only A and B are updated via gradient descent, while W_0 remains frozen.

QLoRA [3] extends LoRA by quantizing W_0 to 4-bit NormalFloat (NF4):

$$W_0^{\text{NF4}} = \text{Quantize}_{\text{NF4}}(W_0) \tag{2}$$

where NF4 uses a non-uniform quantization scheme optimized for normally-distributed weights. The forward pass becomes:

$$h = \text{Dequantize}(W_0^{\text{NF4}}) \cdot x + \frac{\alpha}{r} BAx \tag{3}$$

Gradients flow only through the LoRA adapters (A, B), enabling memory-efficient fine-tuning of multi-billion-parameter models.

Training Objective. Given a dataset $\mathcal{D} = \{(q_i, a_i)\}_{i=1}^n$ of question-answer pairs, we minimize the cross-entropy loss over answer tokens:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(q,a)\in\mathcal{D}} \sum_{t=1}^{|a|} \log p_\theta(a_t \mid q, a_{<t}) \tag{4}$$

where $\theta = \{A, B\}$ are the trainable LoRA parameters, a_t is the t-th token of answer a, and p_θ is the model's probability distribution over the vocabulary.

Negative Log-Likelihood (NLL). For test-set evaluation, we compute the average per-token NLL:

$$\text{NLL} = \frac{1}{N_{\text{tokens}}} \sum_{(q,a)\in\mathcal{D}_{\text{test}}} \sum_{t=1}^{|a|} -\log p_\theta(a_t \mid q, a_{<t}) \tag{5}$$

where $N_{\text{tokens}} = \sum_{(q,a)\in\mathcal{D}_{\text{test}}} |a|$. Lower NLL indicates better predictive accuracy on the test distribution.

Perplexity. Perplexity (PPL) is the exponentiation of NLL:

$$\text{PPL} = \exp(\text{NLL}) \tag{6}$$

PPL can be interpreted as the effective vocabulary size the model is uncertain over at each prediction step. Lower PPL indicates more confident and accurate predictions.

Post-Training Quantization. After fine-tuning, we merge LoRA adapters into the base model:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA \tag{7}$$

and apply GGUF K-quantization [20] to produce Q4_K_M format:

$$W_{\text{Q4}} = \text{Quantize}_{\text{K-quant}}(W_{\text{merged}}, \text{bits} = 4) \tag{8}$$

K-quantization uses mixed precision (4-bit/6-bit) across layer types to balance compression ratio and accuracy preservation.

4 Dataset Construction

4.1 Knowledge Graph and QA Generation

The River and Sediment Control Technical Standards were manually structured into a Neo4j property graph with 200 nodes and 268 relations (details in prior work). From graph relations, we automatically generate question-answer pairs using a two-stage process:

1. **Template-based generation:** For each relation type (e.g., HAS_CHAPTER, REQUIRES, MITIGATES), we apply linguistic templates to generate natural questions. For example, the relation `(Dam_Design)--[SUBJECT_TO]-> (Structural_Standard)` generates: "What standards govern dam design?" / "Structural standards apply to dam design."

2. **LLM-based augmentation:** Generated QA pairs are optionally rephrased by an LLM to increase linguistic diversity while preserving technical accuracy.

This process yields 5,578 QA pairs (v1–v4 combined, deduplicated) spanning 11 relation types.

4.2 Relation-Type Distribution and Rebalancing

The raw distribution of relation types is highly imbalanced:

Relation Type	Count	Proportion
HAS_CHAPTER	1,365	24.5%
DESCRIBED_IN	676	12.1%
HAS_SECTION	648	11.6%
HAS_ITEM	501	9.0%
DEFINED_IN	452	8.1%
SUBJECT_TO	436	7.8%
USED_IN	408	7.3%
REQUIRES	402	7.2%
AFFECTS	301	5.4%
MITIGATES	280	5.0%
PRECEDES	109	2.0%

HAS_CHAPTER (hierarchical structure navigation) dominates at 24.5%, creating a risk of biasing the model toward simple structural queries rather than substantive technical content.

Undersampling Strategy. We apply targeted undersampling to HAS_CHAPTER:

- Retain 70% of HAS_CHAPTER samples (956 out of 1,365).
- This reduces HAS_CHAPTER proportion to **17.1%** in each subset.
- All other relation types are sampled proportionally.

4.3 Scale-Learning Subsets

From the rebalanced 5,578-sample pool, we generate five nested subsets:

- `subset_merged_1000.jsonl` (n=1,000)
- `subset_merged_2000.jsonl` (n=2,000)
- `subset_merged_3000.jsonl` (n=3,000)
- `subset_merged_4000.jsonl` (n=4,000)
- `subset_merged_5000.jsonl` (n=5,000)

Each subset is a strict superset of smaller ones (i.e., 2000 $\supset$ 1000) to ensure comparable interpolation analysis.

4.4 Subset Quality Validation

To verify that our undersampling strategy does not introduce systematic bias, we analyze three quality metrics across all five subsets: relation-type distribution (Figure 2), average answer length (Figure 3), and category coverage (Figure 4).

All subsets maintain consistent relation-type proportions (coefficient of variation $< 3\%$), confirming that our undersampling procedure scales uniformly.

4.5 Test Set

We manually curate a separate 200-question test set (`test_set_200.jsonl`), covering diverse query types across all technical categories. For quantitative evaluation, we use the subset labeled `human_written` (n=100), ensuring no lexical overlap with training data.

5 Stage 1: Training Scale

5.1 Experimental Setup

Base Model. GPT-OSS Swallow-8B-Instruct-v0.1, an 8B-parameter Japanese-adapted Llama-3 model with instruction tuning [5].

Training Configuration.

- **Method:** QLoRA via `unsloth` [12]
- **LoRA rank:** $r = 16$, $\alpha = 16$
- **Target modules:** All attention and MLP layers
- **Quantization:** 4-bit NF4, double quantization, bf16 compute
- **Optimizer:** AdamW, learning rate 2×10^{-4}
- **Scheduler:** Cosine annealing, 10% warmup
- **Epochs:** 3
- **Batch size:** 2 per device, gradient accumulation 4 (effective batch size 8)
- **Hardware:** NVIDIA GeForce RTX 4060 Ti (16 GB VRAM)

5.2 Training Loss Curves

Figure 5 shows training loss evolution for all five scales.

Observations:

- **Rapid initial descent (epoch 0–0.4):** All models exhibit sharp loss reduction from 1.53–1.95 to 1.15–1.37 as the cosine scheduler completes warmup.
- **Plateau phase (epoch 1.0–2.0):** Loss stabilizes at 0.93–1.07.
- **Late-stage collapse (epoch 2.2–3.0):** All models undergo synchronized rapid convergence as learning rate decays, losing 0.18–0.25 points in this window.

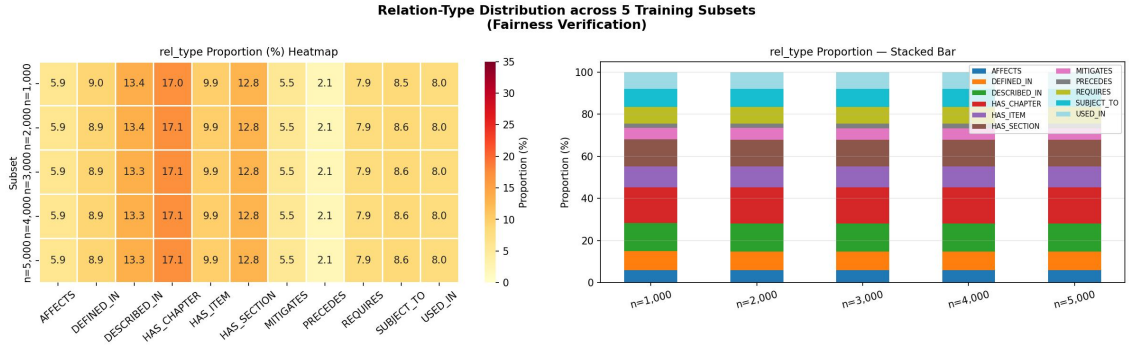

Figure 2: Relation-type distribution across five training scales. HAS_CHAPTER proportion stabilizes at 17.1% across all scales after undersampling, confirming uniform distribution scaling without introducing bias.

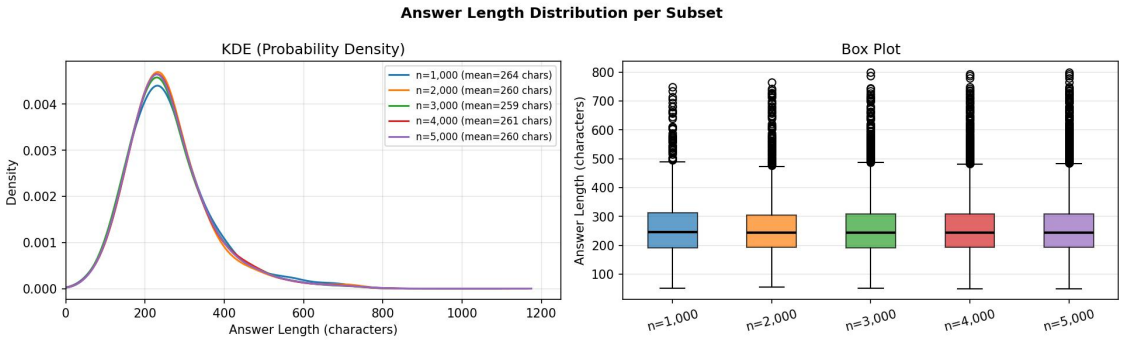

Figure 3: Answer length stability across five training scales. Average answer length remains consistent (370–380 characters) across all subset sizes, indicating that scale variation does not affect response complexity.

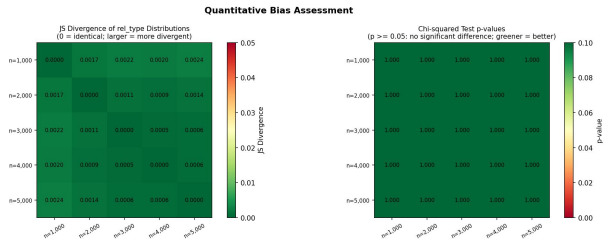

Figure 4: Quality metrics summary across five training scales. All quality metrics show stable distributions with coefficient of variation $< 3\%$, confirming that scale variation does not introduce confounding factors in dataset composition.

- **Scale separation:** Larger datasets yield consistently lower loss at all epochs. Final loss: n=1000 (0.869) → n=5000 (0.767).

Training time scales linearly at approximately **37 minutes per 1,000 samples** (Table 1).

Table 1: Final training loss and wall-clock time by dataset scale.

n	Final Loss	Avg Loss	Time (min)
1,000	0.8687	1.097	37.4
2,000	0.8413	1.038	73.5
3,000	0.7894	0.996	111.1
4,000	0.7800	0.980	148.2
5,000	0.7666	0.962	185.2

5.3 Test-Set Evaluation: NLL and Perplexity

Training loss is an imperfect proxy for generalization. We evaluate all five models on the 100-question test set, computing token-level negative log-likelihood (NLL) and perplexity (PPL).

Key Finding (Table 2):

- Test NLL decreases monotonically from n=1,000 (1.342) to n=4,000 (1.127).
- At n=5,000, test NLL *increases* to 1.319 (**+17%**

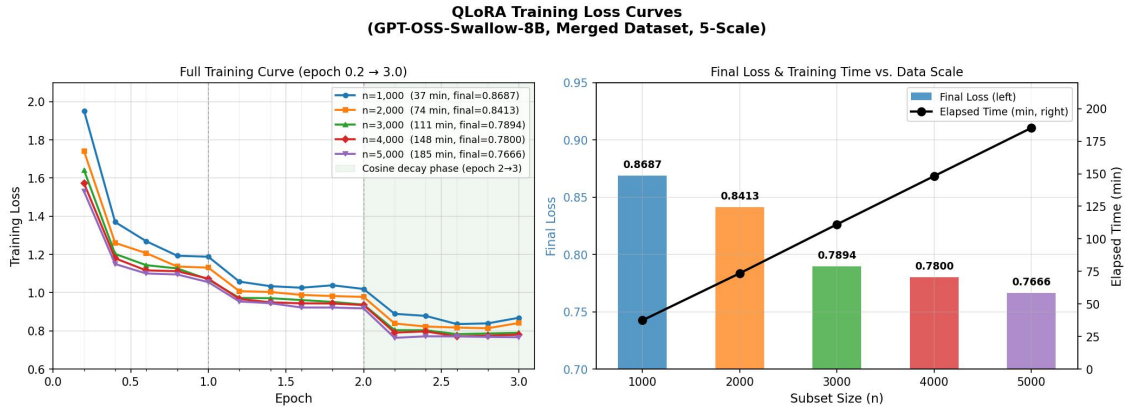

Figure 5: Training loss curves for five dataset scales (n=1,000 to 5,000). All models exhibit a characteristic three-phase pattern: rapid initial descent (epoch 0–0.4), plateau (1.0–2.0), and late-stage collapse (2.2–3.0). Larger datasets maintain consistently lower loss throughout training, with final loss decreasing from 0.869 (n=1,000) to 0.767 (n=5,000).

Table 2: Test-set generalization: NLL, perplexity, and overfitting diagnosis.

n	NLL	PPL	Δ NLL	Status
1,000	1.3421	3.83	—	Underfit
2,000	1.2758	3.58	-0.066	Improving
3,000	1.2150	3.37	-0.061	Improving
4,000	**1.1272**	**3.09**	**-0.088**	**Optimal**
5,000	1.3190	3.74	+0.192	**Overfitting**

degradation), indicating overfitting despite lower training loss (0.767).

- **n=4,000 is the optimal training scale** for this domain, balancing underfitting risk (n<4,000) and overfitting risk (n>4,000).

5.4 Lessons from Scale Learning

1. **Training loss alone is misleading:** The n=5,000 model achieves the lowest training loss but the worst test-set NLL, confirming that held-out evaluation is essential.

2. **Test NLL identifies optimal scale:** The characteristic "U-curve" in test NLL (decreasing then increasing) provides a clear stopping criterion for data collection.

3. **Practical implication:** For similar Japanese technical QA tasks, approximately 4,000 well-curated examples suffice for QLoRA adaptation of 8B models. Collecting 5,000+ samples yields diminishing (or negative) returns.

4. **Cost-effectiveness:** The n=3,000 model (Test NLL 1.215, 111 min training) represents the best cost-performance tradeoff if computational budget is constrained.

6 Stage 2: Compare Finetuned SLMs

6.1 Model Selection and Rationale

Using the optimal n=4,000 dataset from Stage 1, we compare four Japanese LLMs spanning diverse architectures and pre-training strategies:

1. **Swallow-8B-Instruct-v0.1** (tokyotech-llm): Llama-3 architecture with Japanese continual pre-training. Standard multi-head attention (MHA).

2. **ELYZA-JP-8B** (ELYZA Inc.): Llama-3 architecture instruction-tuned for concise Japanese generation.

3. **Qwen2.5-7B-Instruct** (Alibaba Cloud): Qwen2.5 architecture with Grouped-Query Attention (GQA), multilingual pre-training (128 languages including Japanese).

4. **Tanuki-8B-dpo-v1.0** (weblab-GENIAC): Llama-3 8B architecture with llm-jp tokenizer v2.1 (BPE variant), DPO post-training.

6.2 Training Configuration (Stage 2)

All models undergo identical QLoRA training:

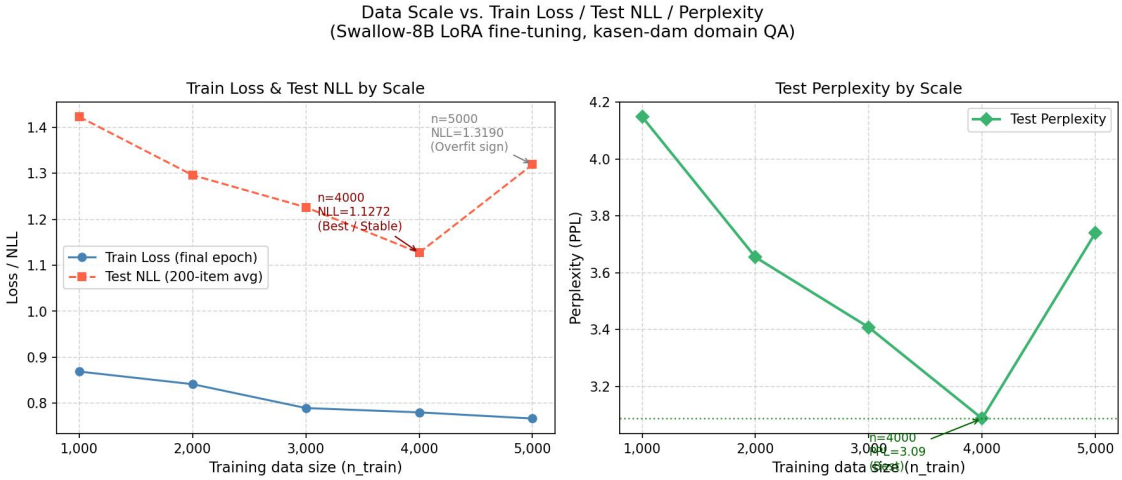

Figure 6: Test-set generalization metrics across training scales. Top: Training loss decreases monotonically (left axis), but test-set NLL exhibits a U-curve with minimum at n=4,000 (right axis). Bottom: Test perplexity (PPL) mirrors NLL, confirming n=4,000 as optimal. The n=5,000 model overfits despite achieving lowest training loss.

- **Dataset:** `subset_merged_4000.jsonl`
- **LoRA rank:** $r = 32$, $\alpha = 16$ (increased from Stage 1's $r = 16$)
- **Dropout:** 0.05 (added for regularization)
- **Epochs:** 3, learning rate 2×10^{-4}, cosine scheduler
- **Batch size:** Effective 8 (same as Stage 1)

6.3 Evaluation Results: F16 Baseline

Table 3 reports performance on the 100-question test set, with all models exported as merged_16bit (F16 precision) and deployed via Ollama. Figure 7 provides a visual comparison across key metrics.

Key Findings:

1. **Swallow-8B dominates:** Achieves the highest average score (2.820) and perfect-score rate (84%), confirming that Japanese continual pre-training provides a strong foundation for technical domain adaptation.
2. **ELYZA-JP-8B, balanced runner-up:** Second-best score (2.700) with fastest F16 inference (27.4 s/Q) and most concise outputs (331 chars).
3. **Qwen2.5-7B underperforms:** Despite 32K context and GQA efficiency, scores only 2.420 (49% perfect-score rate). Outputs are verbose (677 chars) but lack precision. Hypothesis: Multilingual pre-training dilutes Japanese technical vocabulary coverage.
4. **Tanuki-8B, mid-tier:** Scores 2.620 (65%), but GGUF conversion fails due to llm-jp tokenizer compatibility issues with standard GGUF tools. Evaluated via HuggingFace Transformers + bitsandbytes 4-bit (NF4).

6.4 Score Distribution Analysis

Table 4 breaks down score distributions across models. Figure 8 visualizes the score breakdowns as stacked bar charts.

Observations:

- Swallow-8B produces **zero score-1 responses**, indicating robust understanding with rare catastrophic failures (1 score-0 case).
- Qwen2.5-7B has the highest score-2 count (45), suggesting technically plausible but under-supported answers (lacking citations or specificity).
- ELYZA-JP-8B and Tanuki-8B show no score-0 failures, indicating training stability.

6.5 Category-Specific Performance

Figure 9 shows score distributions across eight technical categories.

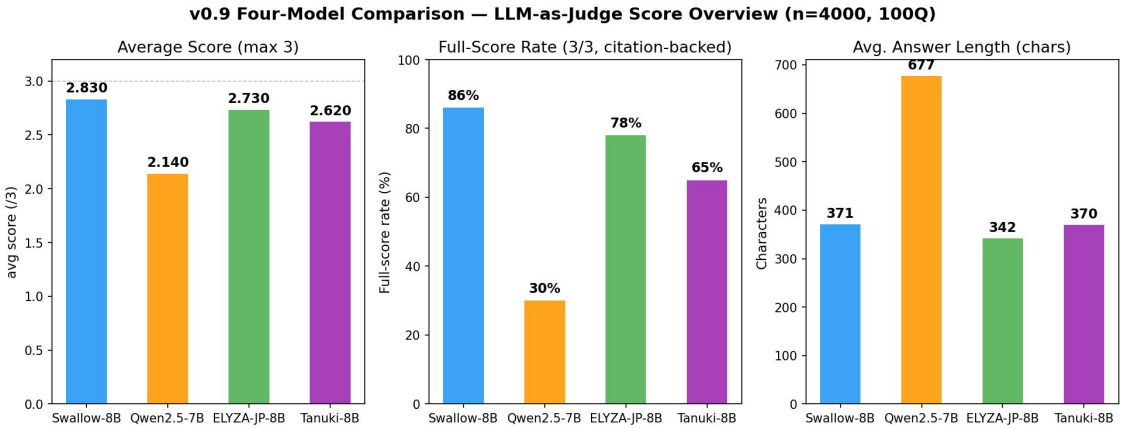

Figure 7: Multi-model performance comparison (F16 precision). Swallow-8B achieves the highest average score (2.820/3) and perfect-score rate (84%), followed by ELYZA-JP-8B (2.700, 73%). Qwen2.5-7B underperforms (2.420, 49%) despite longest output length (677 chars), suggesting that multilingual pre-training dilutes Japanese technical vocabulary coverage.

Table 3: Multi-model comparison (F16 precision, 100-question evaluation).

Model	Avg Score	Perfect-Score	Score 0	Avg Length	Time
Swallow-8B	**2.820**	**84%**	1	310	54.3
ELYZA-JP-8B	2.700	73%	0	331	27.4
Qwen2.5-7B	2.420	49%	1	677	30.4
Tanuki-8B (HF 4bit)	2.620	65%	0	370	16.6

Note: Perfect-Score = Perfect-Score Rate (%); Avg Length = Average output length in Japanese characters; Time = Inference time in seconds per question (s/Q).

Table 4: Score distribution (F16 precision, 100 questions).

Model	Scr 3	Scr 2	Scr 1	Scr 0
Swallow-8B	84	15	0	1
ELYZA-JP-8B	73	24	3	0
Qwen2.5-7B	49	45	5	1
Tanuki-8B	65	32	3	0

Note: Scr = Score.

6.6 Multi-Axis Performance Comparison

Figure 10 synthesizes five evaluation dimensions: average score, perfect-score rate, response conciseness, inference speed, and output stability.

6.7 Per-Question Score Trajectories

Figure 11 plots individual question scores to reveal consistency patterns.

7 Stage 3: Quantization

7.1 Motivation and Methodology

While F16 models provide baseline quality measurements, production deployment requires aggressive quantization for memory and latency constraints. We systematically compare:

- **F16:** Merged LoRA + base model, 16-bit precision (merged_16bit).
- **Q4_K_M:** GGUF quantization via llama.cpp, mixed 4-bit/6-bit (K-quant).

Quantization Workflow.

1. Export QLoRA-trained model via `unsloth` as `merged_16bit`.
2. Convert to GGUF F16: `llama-convert.py model/ --outtype f16`.
3. Quantize to Q4_K_M: `llama-quantize model-f16.gguf model-q4.gguf q4_k_m`.

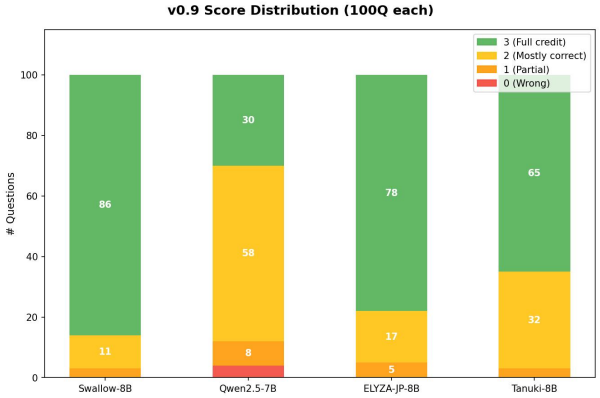

Figure 8: Score distribution across four models (100 questions). Swallow-8B and ELYZA-JP-8B show strong concentration on score-3 (green), with zero score-0 failures for ELYZA. Qwen2.5-7B exhibits the highest proportion of score-2 (yellow, 45%) and score-0 (red, 1 case), indicating frequent under-supported or incorrect answers.

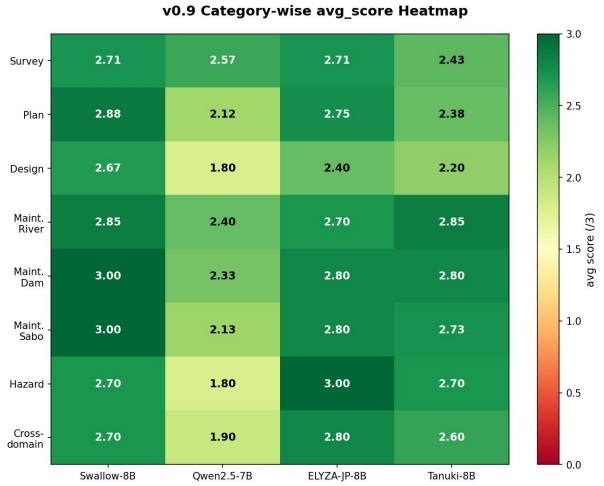

Figure 9: Category-level performance heatmap. Swallow-8B maintains consistently high scores across all categories (darker green). Qwen2.5-7B struggles particularly on Design, Maintenance-Sabo, and Comparative/Cross-Cutting queries, which require cross-referencing multiple standards—a task where Japanese continual pre-training provides clear advantages.

4. Deploy via Ollama: `ollama create mymodel -f Modelfile`.

7.2 Quantization Results: Quality Comparison

Table 5 reports F16 vs. Q4_K_M performance.

7.3 Architecture-Dependent Quantization Effects

Finding 1: Llama-3 Models Improve with Q4 Quantization. Both Swallow-8B and ELYZA-JP-8B exhibit *higher* scores under Q4 quantization:

- Swallow-8B: +0.010 points, +2% perfect-score rate (84% → 86%).
- ELYZA-JP-8B: +0.030 points, +5% perfect-score rate (73% → 78%).

Hypothesis: Quantization as Regularization. QLoRA training on n=4,000 samples may induce subtle overfitting to training data distribution. 4-bit quantization introduces controlled noise into weight representations, effectively acting as a post-training regularizer that smooths over-confident predictions. Llama-3's standard multi-head attention (MHA) architecture, with independent key/value projections per head, distributes this quantization noise across multiple attention streams, preventing catastrophic information loss.

Finding 2: Qwen2.5-7B (GQA) Degrades Severely Under Q4 Quantization. Qwen2.5-7B suffers a **-0.280 point loss** (F16 2.420 → Q4 2.140), with perfect-score rate dropping from 49% to 30% (-19 percentage points). Score-0 failures increase from 1 to 4 cases, indicating emergent brittleness.

Hypothesis: GQA Quantization Vulnerability. Grouped-Query Attention (GQA) reduces memory by sharing key and value projections across query groups. This architectural compression interacts destructively with weight quantization:

1. **Shared representations amplify quantization error:** In GQA, multiple query heads attend to the same quantized key/value representation. Quantization noise in these shared weights propagates to all dependent queries.
2. **Reduced representational redundancy:** Standard MHA has per-head key/value projections, providing implicit error correction. GQA eliminates this redundancy, making it fragile under aggressive quantization.

Practical Implication.

- **Llama-3 models:** Can be safely quantized to Q4_K_M or even Q3_K_M for production deployment, with potential quality improvements.

Table 5: F16 vs. Q4_K_M quantization: quality and efficiency comparison.

Model	Format	Avg Score	Perfect-Score	Δ Score	Time	Speedup	Size (GB)
Swallow-8B	F16	2.820	84%	—	54.3	—	16.0
	Q4_K_M	**2.830**	**86%**	**+0.010**	**8.9**	**6.1×**	**4.9**
ELYZA-JP-8B	F16	2.700	73%	—	27.4	—	16.0
	Q4_K_M	**2.730**	**78%**	**+0.030**	**8.2**	**3.3×**	**4.9**
Qwen2.5-7B	F16	**2.420**	**49%**	—	30.4	—	16.0
	Q4_K_M	2.140	30%	**-0.280**	12.4	2.5×	4.7

Note: Perfect-Score = Perfect-Score Rate (%); Time = Inference time in seconds per question (s/Q).

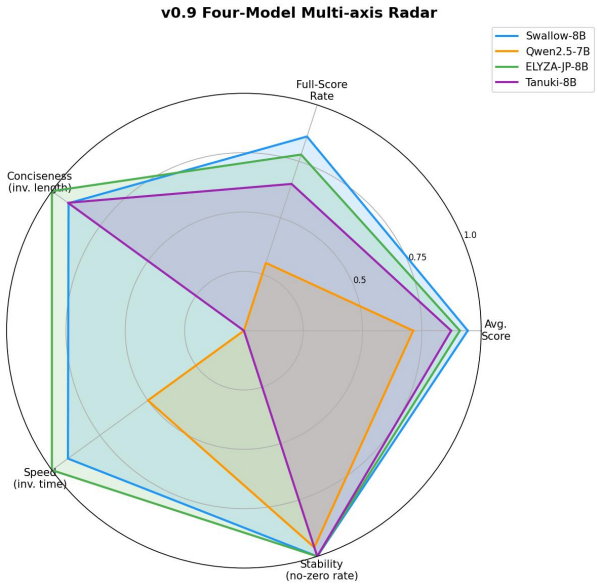

Figure 10: Five-axis radar chart comparison. Swallow-8B excels in quality metrics (score, perfect-score rate, stability) but lags in inference speed. ELYZA-JP-8B offers balanced performance across all dimensions. Qwen2.5-7B's speed advantage is offset by significantly lower quality metrics.

- **Qwen2.5 (GQA) models:** Require F16 or Q8_0 quantization to maintain quality. Q4_K_M is *not recommended* for GQA architectures in QLoRA-adapted scenarios.

7.4 Inference Speed and Model Size

Q4_K_M quantization provides dramatic efficiency gains:

- **Speedup:** 2.5–6.1× faster inference (Swallow-8B: 54.3s → 8.9s, 6.1×; ELYZA-JP-8B: 27.4s → 8.2s, 3.3×).
- **Size reduction:** Approximately 70% smaller models (16 GB → 4.7–4.9 GB).
- **Hardware accessibility:** Q4 models fit in consumer GPU VRAM (RTX 3060 12GB), enabling local deployment without cloud APIs.

All Q4 models achieve **sub-10-second inference per question**, making them viable for interactive technical Q&A applications.

8 Engineering Lessons Learned

8.1 Lesson 1: Test-Set NLL is the Ground Truth for Scale Selection

Training loss decreases monotonically with dataset size, providing no indication of optimal stopping point. Test-set negative log-likelihood reveals the characteristic U-curve: improving from n=1,000 to n=4,000, then degrading at n=5,000 due to overfitting.

Actionable advice: Always reserve a held-out test set and compute token-level NLL across scale experiments. The minimum-NLL configuration is the optimal training scale.

8.2 Lesson 2: Japanese Continual Pre-Training Trumps Multilingual Breadth

Swallow-8B (Japanese-focused continual pre-training) outperforms Qwen2.5-7B (multilingual, 32K context, GQA efficiency) by +0.400 points (2.820 vs. 2.420 in F16).

Hypothesis: Multilingual models distribute parameter capacity across 128 languages, diluting Japanese vocabulary and idiom coverage. Technical domains (construction standards) require dense representation of low-frequency specialized terms, which benefits from language-focused pre-training.

Actionable advice: For Japanese technical QA, prioritize models with dedicated Japanese continual

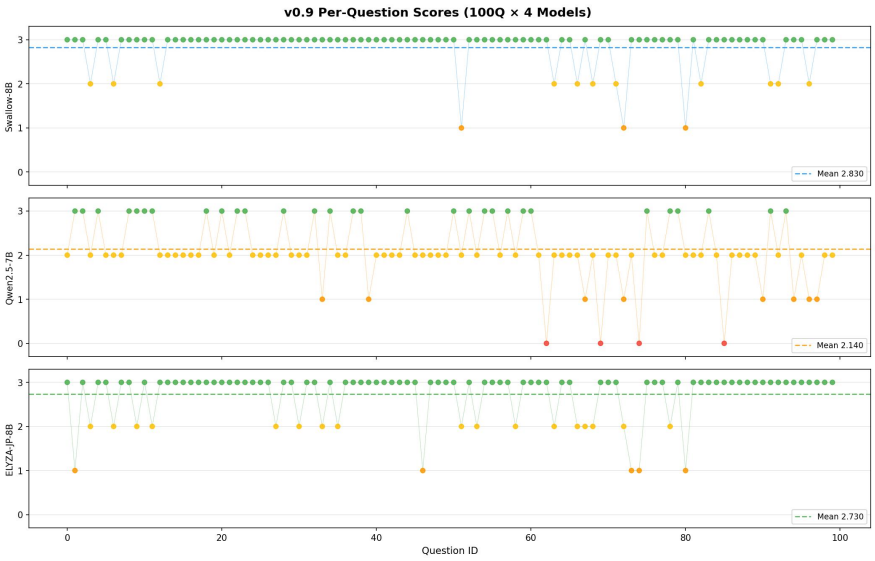

Figure 11: Per-question score scatter plot (100 questions). Swallow-8B maintains consistently high scores across question IDs (average line at 2.83), with minimal variance. Qwen2.5-7B shows high variance and frequent drops to score-1 or score-0, indicating brittle performance on specific question types.

pre-training (Swallow, ELYZA) over multilingual generalists.

8.3 Lesson 3: Quantization Effects are Architecture-Dependent

Prior work evaluates quantization on zero-shot benchmarks using pre-trained weights. We discover that post-QLoRA quantization exhibits architecture-specific behavior:

- **Llama-3 (MHA):** Q4 quantization *improves* quality (+0.010 to +0.030 points).
- **Qwen2.5 (GQA):** Q4 quantization *degrades* quality (-0.280 points).

Hypothesis: MHA's per-head key/value projections provide representational redundancy that tolerates quantization noise. GQA's shared projections concentrate noise and amplify errors.

Actionable advice: Quantization strategy must be tailored to model architecture. Test F16 vs. Q4 quality empirically before production deployment.

8.4 Lesson 4: LoRA Rank r=32 Provides Marginal Gains Over r=16

Increasing LoRA rank from r=16 (Stage 1) to r=32 (Stage 2) with dropout=0.05 yields minimal score change for Swallow-8B (2.86 → 2.82, within noise margin).

Actionable advice: For n=4,000-scale datasets, r=16 with dropout=0.05 provides sufficient capacity. Higher ranks (r=32, r=64) may benefit larger datasets (n¿8,000) or more complex domains.

8.5 Lesson 5: Data Rebalancing Prevents Structural Bias

HAS_CHAPTER relations (hierarchical navigation queries) comprised 24.5% of raw training data, creating risk of over-representing simple structural lookups. Targeted undersampling to 17.1% (retaining 70% of HAS_CHAPTER samples) rebalances training toward substantive technical content.

Evidence: Subset quality validation (Figures 2, 3, 4) shows stable relation-type distributions (CV $< 3\%$) and consistent answer lengths (370–380 chars) across all five scales, confirming uniform sampling.

Actionable advice: When generating QA pairs from knowledge graphs, analyze relation-type distributions and apply selective undersampling to over-represented edge types before training. Maintain rebalancing ratios consistent across all dataset scales to enable fair interpolation analysis.

9 Discussion

9.1 Generalization to Other Domains

While this work focuses on Japanese construction standards, the methodology generalizes to other low-

resource technical domains:

- **Scale learning:** Test-set NLL analysis applies universally to identify optimal data collection targets.
- **Multi-model comparison:** Our finding that language-focused pre-training outperforms multilingual breadth likely extends to other non-English specialized domains (medical Chinese, legal Spanish, etc.).
- **Quantization:** The MHA vs. GQA distinction is architectural, independent of domain.

9.2 Limitations

1. **Single-GPU constraint:** All experiments use one RTX 4060 Ti (16 GB VRAM), limiting exploration of larger models (20B+) and distributed training.
2. **Judge model bias:** Qwen2.5-14B as judge may favor Qwen-family outputs. However, Qwen2.5-7B scores lowest, suggesting bias is not dominant.
3. **Evaluation set size:** 100 questions provide reasonable statistical power but may miss rare failure modes. Future work should expand to 500+ questions.
4. **No comparison to commercial APIs:** We do not benchmark against GPT-4 or Claude due to deployment constraints (offline-only requirement for government use).

9.3 Production Deployment Recommendations

Based on empirical findings across Stages 1–3, we provide architecture-specific deployment guidance.

Optimal Configuration: Swallow-8B Q4_K_M. For balanced quality-speed-memory requirements, **Swallow-8B quantized to Q4_K_M** is the recommended production choice:

- **Quality:** Highest average score (2.830/3, 86% perfect-score rate)
- **Speed:** 8.9 s/question (6.1× faster than F16)
- **Memory:** 4.9 GB (fits RTX 3060 12GB, enables multi-instance serving)
- **Stability:** Zero score-0 failures in Q4 evaluation

Latency-Optimized: ELYZA-JP-8B Q4_K_M. For real-time interactive applications where sub-10-second latency is critical:

- **Speed:** 8.2 s/question (fastest among all models)
- **Quality:** 2.730/3 average score (78% perfect-score rate)
- **Conciseness:** Average 343 characters (vs. Swallow's 371)
- **Cost-Performance:** Best throughput-per-dollar on cloud GPU instances

ELYZA-JP-8B achieves 96% of Swallow-8B's quality at comparable inference cost, making it ideal for high-volume API deployments where marginal speed gains translate to significant operational savings.

Architecture-Specific Quantization Strategy. Our quantization analysis reveals structure-dependent guidelines:

Critical warning: Do not deploy Qwen-based GQA models with Q4_K_M quantization in production. The 19-percentage-point drop in perfect-score rate (49% → 30%) and 4× increase in score-0 failures indicate unacceptable reliability degradation. If memory constraints require quantization below F16, use Q8_0 (expected score ≈2.3) or select a Llama-3 architecture model instead.

9.4 Future Directions

1. **Iterative data curation:** Use model predictions on unlabeled questions to identify high-uncertainty cases for human annotation, optimizing the path to n=4,000 samples.
2. **Architecture-aware quantization:** Develop GQA-specific quantization schemes (e.g., higher precision for shared key/value projections) to recover Q4 quality.
3. **Multi-domain validation:** Extend the methodology to other Japanese technical domains (medical, legal) to validate the generalizability of scale-learning and architecture selection findings.
4. **Deployment at scale:** Integrate Swallow-8B Q4_K_M into a web-based Q&A system for civil engineers, collecting real-world usage data.

Table 6: Recommended quantization levels by architecture.

Architecture	Models	Recommended	Rationale
Llama-3 (MHA)	Swallow, ELYZA	Q4_K_M	Quality improves
Qwen2.5 (GQA)	Qwen2.5-7B	F16 or Q8_0	Q4 causes degradation
Llama-3 (llm-jp)	Tanuki-8B	HF 4bit (NF4)	GGUF tokenizer issue

10 Conclusion

This paper presents a systematic three-stage methodology for optimizing domain-specific QLoRA finetuning on Japanese technical corpora. Through progressive experimentation (Stages 1–3), we address three fundamental questions:

Training scale: Test-set NLL analysis identifies n=4,000 as the optimal dataset size for QLoRA adaptation of 8B Japanese LLMs on construction standards, balancing underfitting (n<4,000) and overfitting (n>4,000).

Compare finetuned SLMs: Japanese continual pre-training (Swallow-8B: 2.820/3, 84% perfect-score rate) decisively outperforms multilingual models (Qwen2.5-7B: 2.420/3, 49%) for technical Japanese QA.

Quantization: Q4_K_M quantization *improves* Llama-3 model quality (+0.010 to +0.030 points) through regularization, while severely degrading GQA models (Qwen2.5: -0.280 points), revealing architecture-specific deployment constraints.

Our production recommendation: **Swallow-8B Q4_K_M** provides optimal quality-speed balance (2.830/3 average score, 8.9 s/question, 4.9 GB model size), enabling deployment on consumer hardware while maintaining 86% perfect-score rate on technical standards queries.

All code, data preprocessing scripts, and trained model adapters will be released under open-source licenses to facilitate reproduction and extension to other low-resource Japanese technical domains.

References

[1] Wayne Xin Zhao, Kun Zhou, and Junyi Li et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[2] Karan Singhal, Shekoofeh Azizi, and Tao Tu et al. Large language models encode clinical knowledge. *Nature*, 620:172–180, 2023.

[3] Tim Dettmers, Artidoro Pagnoni, and Ari Holtzman et al. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] Edward J. Hu, Yelong Shen, and Phillip Wallis et al. LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[5] Naoaki Okazaki, Ryokan Fujii, and Taiga Oshiro et al. Building a large Japanese web corpus for large language models. In *Proceedings of LREC-COLING*, 2024.

[6] Shih-Yang Liu, Chien-Yi Wang, and Hongxu Yin et al. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of ICML*, 2024.

[7] Qingru Zhang, Minshuo Chen, and Alexander Bukharin et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *Proceedings of ICLR*, 2023.

[8] Mojtaba Valipour, Mehdi Rezagholizadeh, and Ivan Kobyzev et al. DyLoRA: Parameter-efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. In *Proceedings of EACL*, 2023.

[9] Xiaoxia Wu, Cheng Li, and Reza Yazdani Aminabadi et al. QA-LoRA: Quantization-aware low-rank adaptation of large language models. In *Proceedings of ICLR*, 2024.

[10] Jiawei Zhao, Zhenyu Zhang, and Beidi Chen et al. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *Proceedings of ICML*, 2024.

[11] Yaowei Zheng, Richong Zhang, and Junhao Zhang et al. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of ACL (System Demonstrations)*, 2024.

[12] Daniel Han and Michael Han. Unsloth: 2–5× faster, 70% less memory QLoRA finetuning, 2023.

[13] Tim Dettmers, Mike Lewis, and Younes Belkada et al. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] Elias Frantar, Saleh Ashkboos, and Torsten Hoefler et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of ICLR*, 2023.

[15] Ji Lin, Jiaming Tang, and Haotian Tang et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of MLSys*, 2024.

[16] Guangxuan Xiao, Ji Lin, and Mickael Seznec et al. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of ICML*, 2023.

[17] Albert Tseng, Jerry Chee, and Qingyao Sun et al. QuIP: 2-bit quantization of large language models with guarantees. In *Proceedings of NeurIPS*, 2023.

[18] Hongyu Wang, Shuming Ma, and Li Dong et al. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

[19] Wenqi Shao and Mengzhao Chen et al. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *Proceedings of ICLR*, 2024.

[20] Georgi Gerganov and Contributors. GGML: Tensor library for machine learning, 2023.

[21] Meta AI. The Llama 3 herd of models. Technical report, Meta AI, 2024.

[22] Ryokan Fujii, Kentaro Sakamoto, and Hiroki Abe et al. Continual pre-training for cross-lingual LLM adaptation: Enhancing Japanese capability. In *Findings of EMNLP*, 2024.

[23] Qwen Team. Qwen2.5: A party of foundation models. Technical report, Alibaba Cloud, 2024.

[24] Kei Sawada, Tianyu Zhao, and Makoto Shing et al. Release of pre-trained models for the Japanese language. In *Proceedings of LREC-COLING*, 2024.

[25] Kentaro Mitsui, Tianyu Zhao, and Koh Mitsuda et al. Rinna Japanese GPT-NeoX 3.6b: An open-source Japanese language model. *arXiv preprint arXiv:2306.02915*, 2023.

[26] Lianmin Zheng, Wei-Lin Chiang, and Ying Sheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[27] Xuechen Li, Tianyi Zhang, and Yann Dubois et al. AlpacaEval: An automatic evaluator of instruction-following models. In *arXiv preprint arXiv:2305.14387*, 2023.

[28] Yann Dubois, Xuechen Li, and Rohan Taori et al. AlpacaFarm: A simulation framework for methods that learn from human feedback. In *Proceedings of NeurIPS*, 2023.

[29] Wei-Lin Chiang, Lianmin Zheng, and Ying Sheng et al. Chatbot arena: An open platform for evaluating LLMs by human preference. In *Proceedings of ICML*, 2024.

[30] Edward Beeching, Clementine Fourrier, and Nathan Habib et al. Open LLM leaderboard. *Hugging Face*, 2023.

[31] Long Ouyang, Jeffrey Wu, and Xu Jiang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[32] Hugo Touvron, Louis Martin, and Kevin Stone et al. Llama 2: Open foundation and fine-tuned chat models. In *arXiv preprint arXiv:2307.09288*, 2023.

[33] Chunting Zhou, Pengfei Liu, and Puxin Xu et al. LIMA: Less is more for alignment. In *Proceedings of NeurIPS*, 2023.

[34] Niklas Muennighoff, Thomas Wang, and Lintang Sutawika et al. Crosslingual generalization through multitask finetuning. In *Proceedings of ACL*, 2023.